Machine learning: Polynomial Regression (Hồi quy đa thức)

2.Hồi quy đa thức

- Nếu dữ liệu không phù hợp với hồi quy tuyến tính (các điểm không đi qua đường thẳng nhưng có thể đi qua một đường cong nào đó), có thể cân nhắc sử dụng hồi quy đa thức)
- Sử dụng mối quan hệ giữa các biến x và y để tìm ra cách tốt nhất để vẽ một đường đi qua các điểm dữ liệu (chọn bậc đa thức sao cho tốt nhất)

2) Polynomial Regression

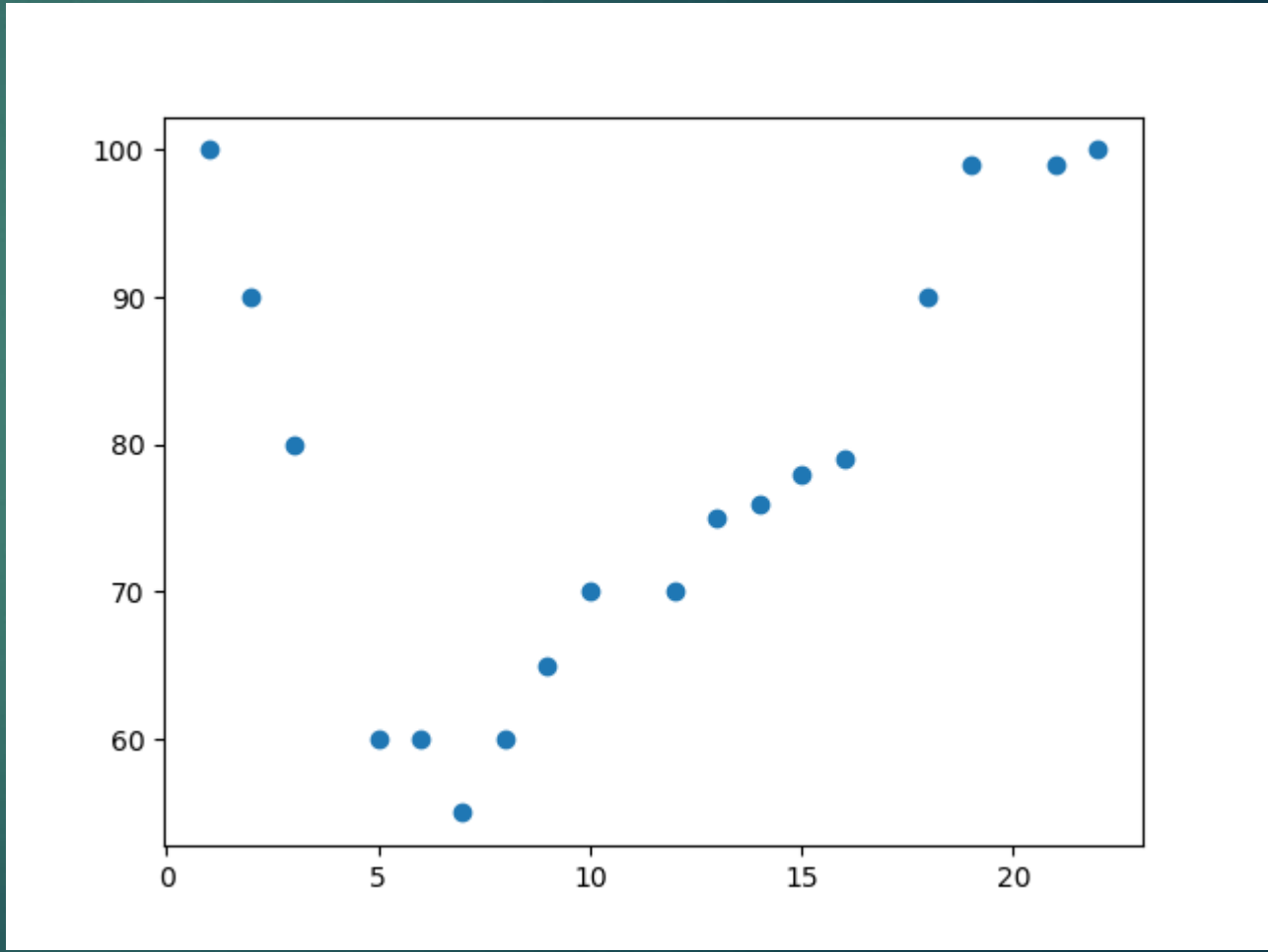
Khái niệm: Mở rộng đặc trưng bậc d: $[x, x^2, \ldots, x^d]$ rồi hồi quy tuyến tính.

Cách dùng: `PolynomialFeatures` + chuẩn hóa; chọn bậc bằng cross-validation.

Chỉ số: MAE, RMSE, R^2; so sánh theo **degree**.

Hồi quy đa thức: Như thế nào

- Ví dụ tìm mối quan hệ giữa các thời điểm trong ngày đi qua 1 địa điểm và tốc độ của các ô tô.
- x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
  y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]



Hồi quy đa thức: Như thế nào


```
import numpy
from sklearn.metrics import r2_score

x
= [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,2
2]
y
= [100,90,80,60,60,55,60,65,70,70,75,76,78,79,9
0,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

print(r2_score(y, mymodel(x)))
```

numpy.polyfit(x, y, 3): Hàm polyfit của NumPy được sử dụng để thực hiện việc tìm hệ số của một đa thức dựa trên dữ liệu đã cho. Trong trường hợp này, các tham số của hàm là x, y và 3.

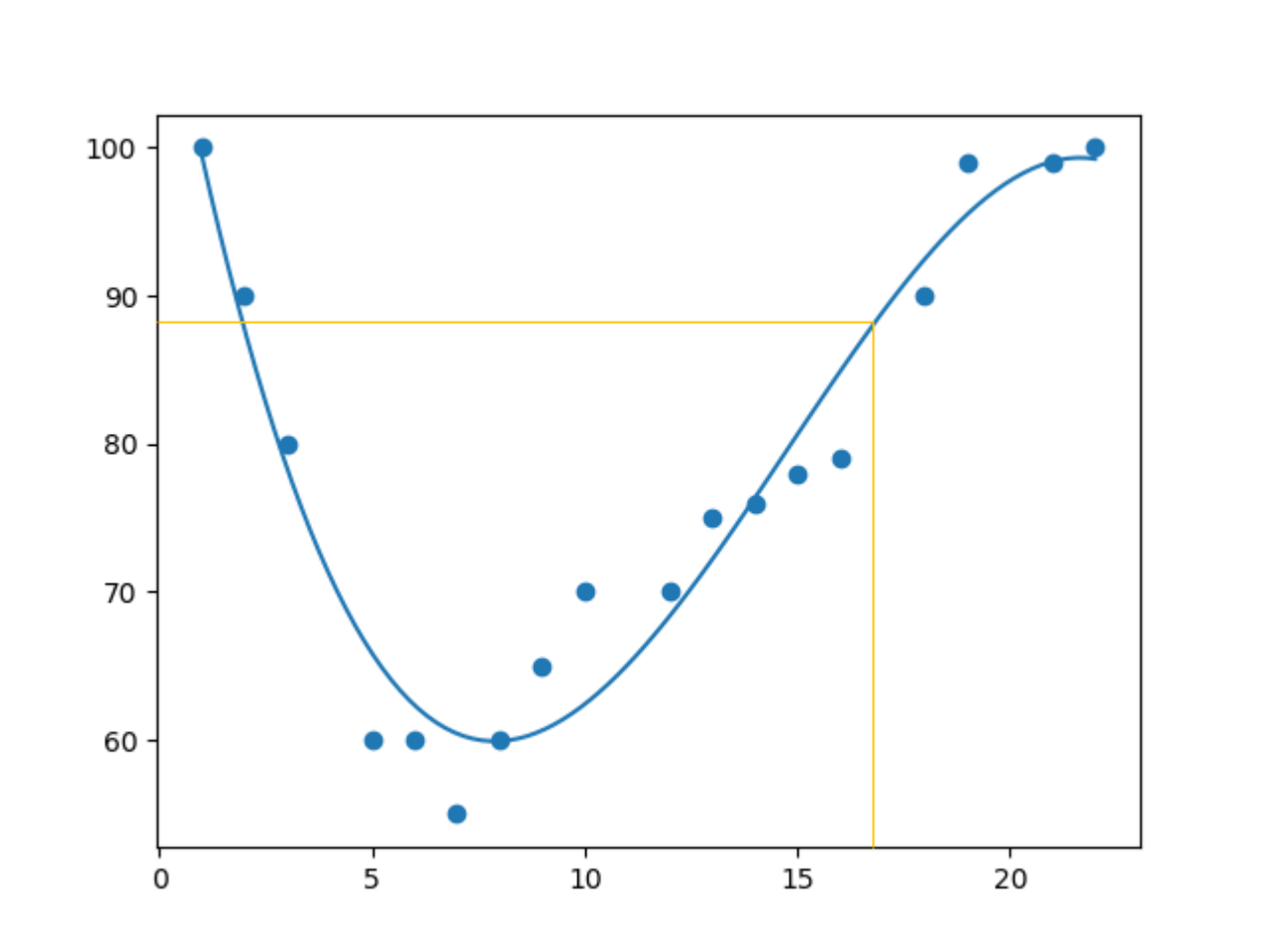
```
r2_score: 0.94
```

Giá trị r2_score nằm trong khoảng từ 0 tới 1, 0: không liên quan, 1: 100% liên quan

Hồi quy đa thức: Dự báo


```
import numpy
from sklearn.metrics import r2_score
x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
speed = mymodel(17)
print(speed)
```

```
speed 88.87331269698001
```



Hồi quy đa thức: chọn bậc đa thức

rng = np.random.RandomState(42)

X = np.sort(5 * rng.rand(200, 1) - 2.5, axis=0)

y = np.sinc(X).ravel() + 0.1 * rng.randn(200)

- Sinh **200 điểm dữ liệu** ngẫu nhiên trong khoảng $[-2.5, 2.5]$.
- Hàm mục tiêu: $y = \mathrm{sinc}(x) + \text{nhiễu}$.
  (trong đó $\mathrm{sinc}(x) = \sin(\pi x)/(\pi x)$).

```python
for d in [1, 3, 5]:
    model = Pipeline([
        ("poly",   PolynomialFeatures(degree=d, include_bias=False)),
        ("scaler", StandardScaler()),
        ("linreg", LinearRegression())
    ]).fit(X_train, y_train)

    pred = model.predict(X_test)
    rmse = rmse_np(y_test, pred)
    r2   = r2_score(y_test, pred)
    print(f"Degree={d} -> RMSE={rmse:.4f}, R2={r2:.4f}")
```

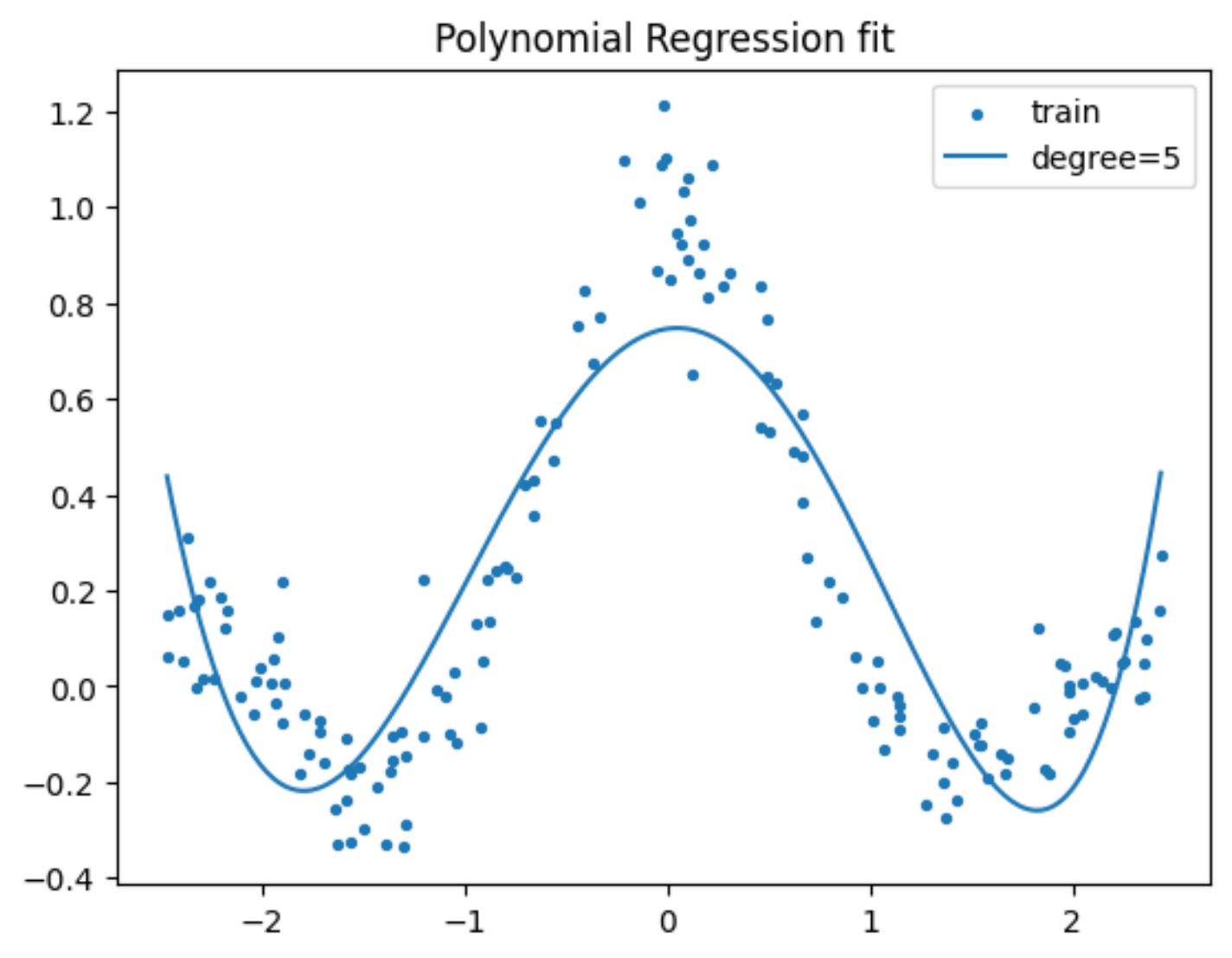
- Với mỗi **bậc đa thức (degree = 1, 3, 5)**:
  1. **PolynomialFeatures**: mở rộng đặc trưng $X \to [x, x^2, \ldots, x^d]$.
  2. **StandardScaler**: chuẩn hóa các đặc trưng.
  3. **LinearRegression**: học mô hình tuyến tính trên các đặc trưng đã mở rộng.
- In ra RMSE và R^2 để so sánh mức độ khớp dữ liệu.

Hồi quy đa thức: chọn bậc đa thức

Degree=1 -> RMSE=0.3352, R2=-0.0128

Degree=3 -> RMSE=0.3028, R2=0.1734

Degree=5 -> RMSE=0.1996, R2=0.6409



StandardScaler: chuẩn hóa đặc trưng

`StandardScaler` trong scikit-learn thực hiện phép biến đổi:

$$x' = \frac{x - \mu}{\sigma}$$

- x: giá trị gốc của đặc trưng.
- μ: giá trị trung bình của đặc trưng đó trong tập train.
- σ: độ lệch chuẩn (standard deviation).
- x': giá trị sau khi chuẩn hóa.

Ý nghĩa:

- Sau khi chuẩn hóa, **mỗi đặc trưng có trung bình = 0 và phương sai = 1**.
- Khi ta tạo đặc trưng đa thức ($x, x^2, x^3, \ldots$), giá trị có thể rất lớn (ví dụ: x^5 dễ vượt trội so với x).
- `StandardScaler` giúp đưa chúng về cùng thang đo để mô hình **học ổn định hơn** và tránh hiện tượng **một số đặc trưng chiếm ưu thế tuyệt đối**.

k-fold cross-Validation: chọn bậc tối ưu

- **Cross-validation (CV)** là cách đánh giá mô hình bằng cách **chia dữ liệu thành nhiều phần (folds)** thay vì chỉ train-test split một lần.
- **5-fold CV** nghĩa là:
  - Tập dữ liệu được chia đều thành **5 phần bằng nhau** (gọi là fold).
  - Với mỗi lần lặp:
    - Chọn **1 fold làm tập kiểm thử (validation)**.
    - Dùng **4 fold còn lại để huấn luyện**.
  - Quá trình này lặp lại **5 lần**, đảm bảo rằng **mỗi điểm dữ liệu đều được dùng để kiểm thử đúng 1 lần**.

Cách làm:

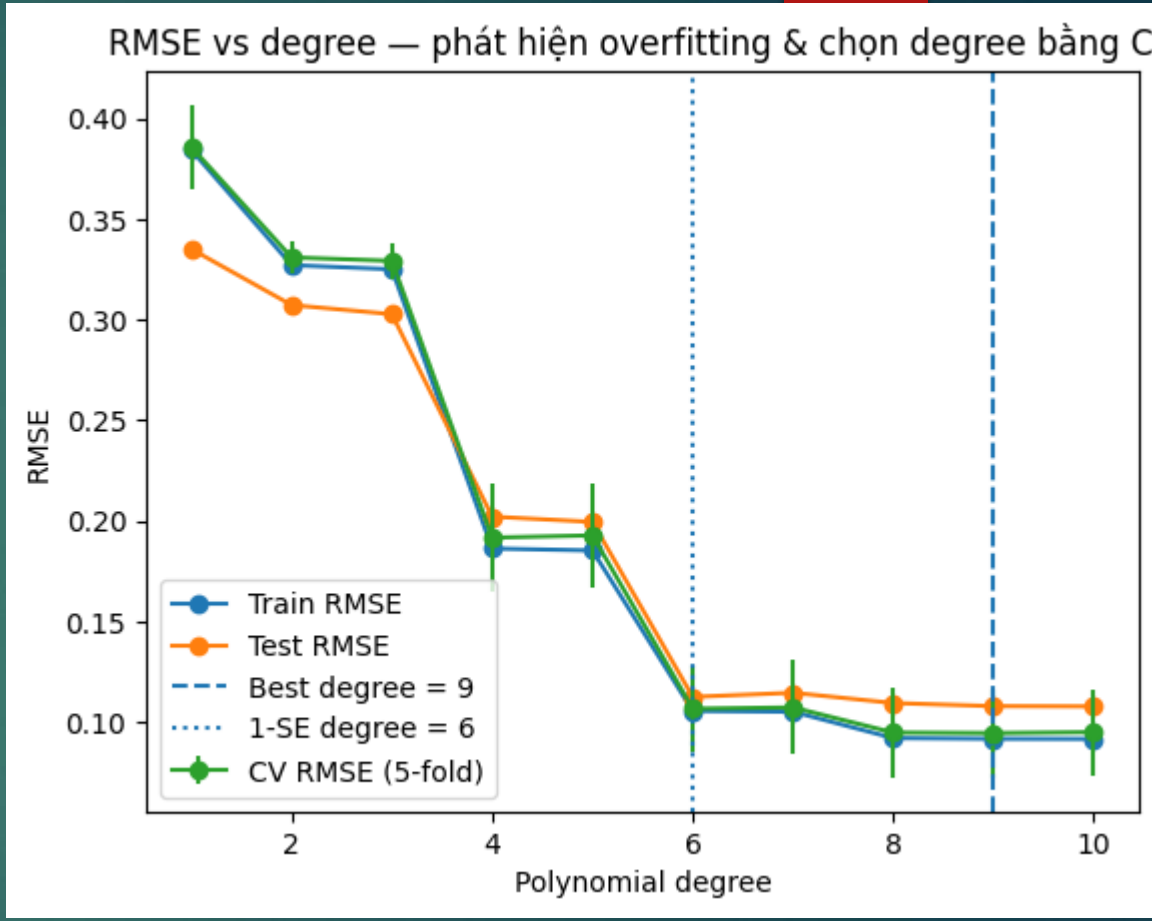
•Dùng `KFold` hoặc `cross_val_score`.
•Đặt pipeline giống cũ.
•Với mỗi degree, tính điểm CV (ví dụ: RMSE hoặc R^2 trung bình).
•Chọn degree có điểm tốt nhất.

```
# Chọn degree tối ưu theo CV-mean nhỏ nhất
best_idx = int(np.argmin(cv_mean))
best_deg = list(degrees)[best_idx]

# Quy tắc 1-SE: chọn degree nhỏ nhất có CV-mean <= (min + std của degree tốt nhất)
one_se_threshold = cv_mean[best_idx] + cv_std[best_idx]
deg_1se = next(d for d, m in zip(degrees, cv_mean) if m <= one_se_threshold)
```

k-fold cross-Validation: chọn bậc tối ưu

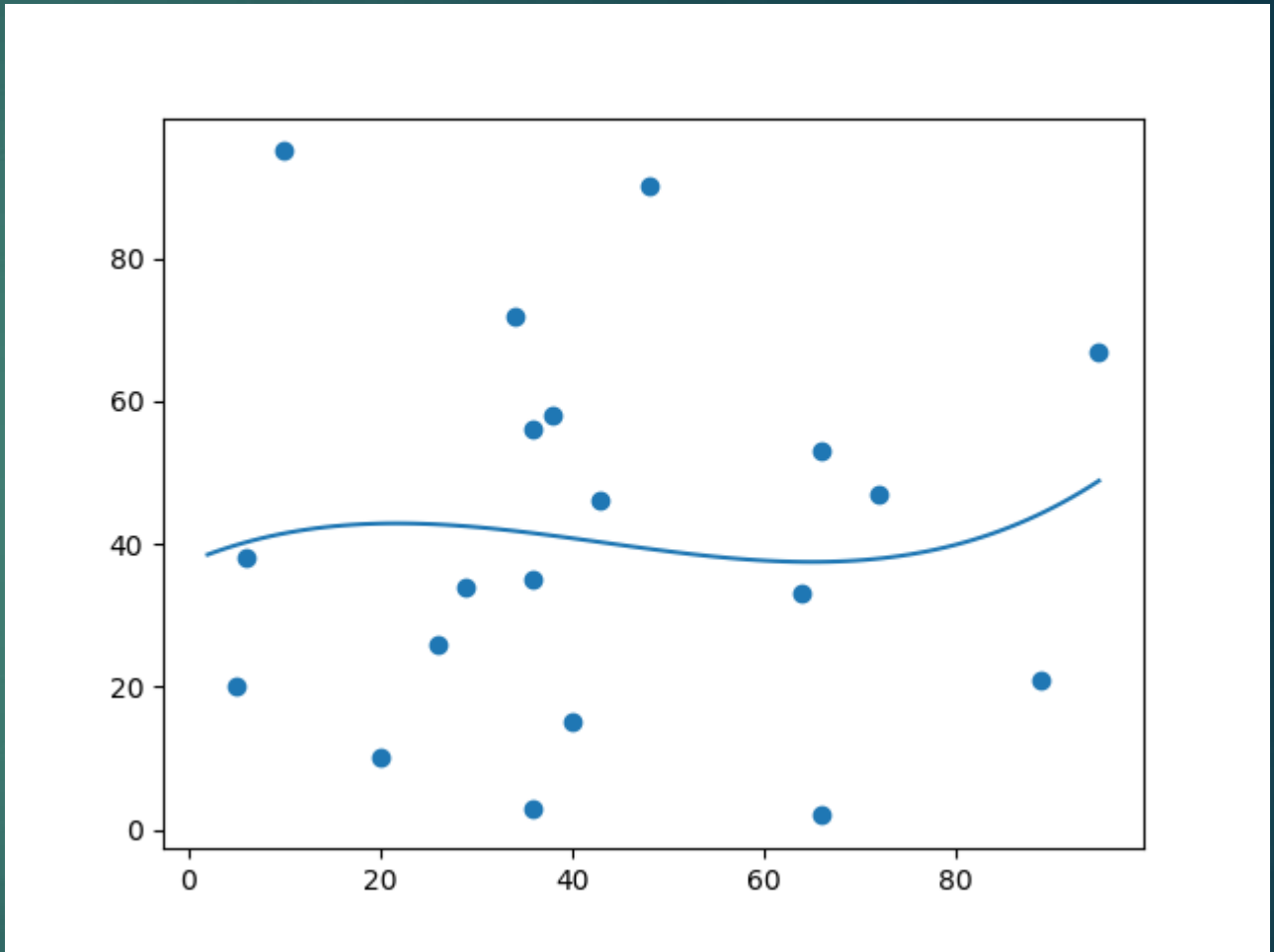

```
eg | Train RMSE | Test RMSE | Test R2 | CV RMSE
(mean±std) 1 | 0.3844 | 0.3352 | -0.0128 |
0.3859 ± 0.0209 2 | 0.3272 | 0.3073 | 0.1485 |
0.3310 ± 0.0079 3 | 0.3251 | 0.3028 | 0.1734 |
0.3293 ± 0.0092 4 | 0.1864 | 0.2022 | 0.6314 |
0.1918 ± 0.0273 5 | 0.1854 | 0.1996 | 0.6409 |
0.1929 ± 0.0262 6 | 0.1056 | 0.1126 | 0.8856 |
0.1068 ± 0.0213 7 | 0.1052 | 0.1147 | 0.8814 |
0.1074 ± 0.0234 8 | 0.0921 | 0.1096 | 0.8918 |
0.0949 ± 0.0224 9 | 0.0918 | 0.1080 | 0.8948 |
0.0946 ± 0.0220 10 | 0.0917 | 0.1079 | 0.8950 |
0.0951 ± 0.0216 Best degree (min CV RMSE): 9 1-
SE rule degree (đơn giản hơn, gần tối ưu): 6
```



Hồi quy đa thức: Bad fit


```
x
= [89,43,36,36,95,10,66,34,38,20,26,2
9,48,64,6,5,36,66,72,40]
y
= [21,46,3,35,67,95,53,72,58,10,26,34
,90,33,38,20,56,2,47,15]
```

r2_score:
0.009952707566680652



Bài tập

- Với bộ dữ liệu auto-mpg.csv, mô tả dữ liệu tại Auto MPG - UCI Machine Learning Repository; sử dụng hồi quy đa thức để Nhận diện quan hệ phi tuyến và mô hình hoá bằng **Polynomial Regression** (ví dụ dự báo mpg từ horsepower):
  - Vẽ scatter plot (mpg và horsepower), nhận xét mối quan hệ giữa 2 đại lượng này
  - Chia dữ liệu: train:valid:test
  - So sánh RMSE theo degree = 1..10 & phát hiện overfitting
  - Fit mô hình theo degree đã chọn & chẩn đoán
  - Viết báo cáo tổng kết

Machine learning: Multiple Regression (Đa hồi quy)

Đa hồi quy

- Hồi quy bội giống như hồi quy tuyến tính, nhưng có nhiều hơn một giá trị độc lập, nghĩa là chúng ta cố gắng dự đoán một giá trị dựa trên hai biến trở lên
- Ví dụ dự báo lượng khí thải ô tô dựa vào volume và weight

Car	Model	Volume	Weight	CO2
BMW	1	1600	1365	99
Mazda	3	2200	1280	104
Skoda	Rapid	1600	1119	104
Ford	Focus	2000	1328	105
Ford	Mondeo	1600	1584	94
Opel	Insignia	2000	1428	99
Mercedes	C-Class	2100	1365	99
Skoda	Octavia	1600	1415	99
Volvo	S60	2000	1415	99

Multiple Regression

Khái niệm:

Mở rộng từ hồi quy tuyến tính đơn biến (một biến độc lập) sang nhiều biến độc lập. Mô hình có dạng:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

trong đó $x_1, x_2, \ldots, x_p$ là các biến dự đoán.

Đa hồi quy


```
import pandas
from sklearn import linear_model

df = pandas.read_csv("data.csv")

X = df[['Weight', 'Volume']]
y = df['CO2']

regr = linear_model.LinearRegression()
regr.fit(X, y)

#predict the CO2 emission of a car where the weight is
2300kg, and the volume is 1300cm3:
predictedCO2 = regr.predict([[2300, 1300]])

print(predictedCO2)
```

```
[107.2087328]
```

Đa hồi quy: hệ số

```
import pandas
from sklearn import linear_model

df = pandas.read_csv("data.csv")

X = df[['Weight', 'Volume']]
y = df['CO2']

regr = linear_model.LinearRegression()
regr.fit(X, y)

print(regr.coef_)
```

Weight: 0.00755095
Volume: 0.00780526

```
[0.00755095 0.00780526]
```

These values tell us that if the weight increase by 1kg, the CO_2 emission increases by 0.00755095g.
And if the engine size (Volume) increases by 1 cm^3, the CO_2 emission increases by 0.00780526 g.

Đa hồi quy


```
import pandas as pd
from google.colab import files
uploaded=files.upload()
import io
df = pd.read_csv(io.BytesIO(uploaded['data.csv’]))
X = df[['Weight', 'Volume']]
y = df['CO2']
print (df)
from sklearn import linear_model
regr = linear_model.LinearRegression()
regr.fit(X, y)
predictedCO2 = regr.predict([[2300, 1300]])
print(predictedCO2)
```

```
[107.2087328]
```

Bài tập

- Sử dụng hồi quy tuyến tính để dự báo giá nhà trên bộ dữ liệu được mô tả sau:
  The California housing dataset — Scikit-learn course

  Các công việc cần thực hiện:

  1. Báo cáo tổng quan dữ liệu
  2. Làm chuẩn dữ liệu (nếu có)
  3. Chia train/test (80/20, random_state=42).
  4. Sử dụng huấn luyện bằng LinearRegression.
  5. Báo cáo MAE, RMSE, R^2, r.
  6. Nhận xét kết quả và mô hình.

```
# ---- Hàm chỉ số (NumPy) ----
def mae_np(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    return np.mean(np.abs(y_true - y_pred))

def rmse_np(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    return np.sqrt(np.mean((y_true - y_pred) ** 2))

def r2_np(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    sse = np.sum((y_true - y_pred) ** 2)
    sst = np.sum((y_true - np.mean(y_true)) ** 2)
    return 1.0 - sse / sst if sst > 0 else np.nan

def pearson_r_np(y_true, y_pred):
    """Hệ số tương quan Pearson r giữa y và ŷ."""
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    yt = y_true - y_true.mean()
    yp = y_pred - y_pred.mean()
    denom = np.sqrt((yt**2).sum()) * np.sqrt((yp**2).sum())
    return (yt @ yp) / denom if denom > 0 else np.nan
```